

AgentGEM: Generative Environments Maker for Scalable Agentic RL

Anonymous ACL submission

Abstract

Reinforcement learning (RL) for language agents requires *interactive* environments in which tool calls yield observations, and *verifiable* rewards can be computed reliably. We present **AgentGEM**, a scalable environment generator that synthesizes *verifiable* agentic tasks and packages each task into an isolated sandbox suitable for interaction and verification. Specifically, a **General Agent** that (1) seeds a sandbox with realistic data; (2) synthesizes task-specific tools with checkable schemas; and (3) closes the loop by executing a reference verifier and repairing failures; a **Search Agent** that constructs multi-hop question answering tasks via web search, visiting, and filtering; and a **CLI Agent** that mines realistic GitHub repositories and produces function-implementation tasks equipped with executable tests validated inside a secure sandbox. Across pipelines, AgentGEM emphasizes a unified task schema, tool interfaces, and execution-based validation. Empirically, AgentGEM achieves ... average improvement on eight challenging agent benchmarks, consistently outperforms strong agents based on frontier models and prior baselines.

1 Introduction

Language agents are increasingly deployed to solve problems where essential information is not available in context but must be acquired through interaction with external tools, such as web search, database querying, and code execution. In such scenarios, agents must integrate planning with tool orchestration, interpret observations, and revise their beliefs in the presence of context rot. Agentic reinforcement learning (RL) is an emerging paradigm for improving agent capabilities. However, the primary bottleneck in scaling agentic RL is not the learning algorithm, but the availability of interactive environments that provide grounded observations and reliable verification.

Existing resources typically occupy only a fraction of this design space. Static datasets and supervised traces provide breadth but collapse interaction into offline contexts. Interactive benchmarks expose tool use, yet rely on underspecified success criteria, fragile evaluators, and limited domain coverage. Synthetic tasks can be generated at scale, but are frequently vulnerable to reward hacking: if verification is weak, a policy can exploit evaluator shortcuts rather than learn grounded tool use. Moreover, even when verifiers are available, the tool interface can be unrealistic (*e.g.*, allowing unconstrained access to the database), leading to agents that cannot transfer to deployment scenarios. These limitations create a trade-off between diversity, realism, and verifiability, which, in turn, constrains the effectiveness of reinforcement learning for language agents.

As such, we introduce AgentGEM, a generative environment maker that addresses these issues through reliable output validation and principled tool interface design for agentic RL. AgentGEM treats environment generation as a compilation process producing verifiable artifacts. For each executable environment, it generates three components: a task specification, a tool interface governed by checkable schemas, and verification artifacts that yield a clear accept-or-reject decision under sandbox execution. These components run in an isolated sandbox, where quality gates filter out degenerate instances such as empty outputs or trivially simple interactions. When generation goes wrong, AgentGEM repairs rather than restarts. If execution fails or verification is insufficient, it enters a closed-loop repair cycle, regenerating only the failing component instead of the entire environment. Where capabilities are missing, it selectively expands the tool interface while preserving safety constraints throughout.

AgentGEM implements three complementary pipelines that instantiate this philosophy across

common tool-using regimes. The data-grounded pipeline generates tasks over sandbox-local resources by synthesizing task-specific tools and pairing them with executable validation and repair. The web-grounded pipeline constructs multi-hop question answering instances by explicitly collecting evidence through search and browsing and filtering instances through verification, with an optional hardness constraint that retains only those instances that defeat model-generated candidate answers. The repository-grounded pipeline produces function-implementation tasks in real codebases and anchors correctness in test execution, yielding a natural execution-based reward signal. Collectively, these pipelines provide a practical substrate for large-scale training and evaluation of tool-using agents, while preserving traceability and reproducibility of generated instances.

2 Preliminaries

We study environment generation for tool-using agents under sandboxed execution. An environment instance specifies (i) a natural-language task instruction, (ii) a constrained tool interface with machine-checkable schemas, and (iii) a verification mechanism that yields an unambiguous accept/reject decision. AgentGEM supports two instance families: executable tasks whose correctness is decided by sandbox execution (via structured output validation and an acceptance procedure), and web-grounded question-answer records whose correctness is decided by verifying the answer against explicitly collected evidence. A generated instance is considered valid only if it satisfies its corresponding verification criterion and passes additional quality constraints that discourage degenerate behavior (e.g., empty outputs or trivial interaction). Given a fixed generation budget in model calls and sandbox executions, the objective is to maximize the yield of valid instances while preserving diversity across topics, tools, and interaction patterns.

3 AgentGEM: Synthesizing, Scaling, and Evolving RL Environments

Inspired by the challenges of *scalable, realism and continuous self-improvement* for general-purpose agents outlined in the background, we present a comprehensive methodology AgentGEM for constructing a scalable, realistic training ecosystem and a self-evolving agent training loop. Our

approach consists of two tightly coupled, iterative components: (1) **Agentic Environment-Task Discovery**, which autonomously mines real-world data and synthesizes executable, verifiable environments and tasks; and (2) **Continuous Self-Evolving Agent Training**, which leverages these environments for multi-agent reinforcement learning and uses a dynamic arena to diagnose weaknesses and drive targeted co-evolution of agents and environments.

3.1 Agentic Environment-Task Discovery

3.1.1 General Agent

Stage 1: Autonomous Database Seeding The foundation of a realistic environment is a topic-aligned, structured database. Manually curating such databases is infeasible at scale. Therefore, we design an autonomous data pipeline that seeds databases from the web.

For a set of environment themes or topics M , given $m \in M$, the pipeline first scans a local sandbox directory for existing structured data files (e.g., .json, .jsonl) and performs a fast file profiling without deep analysis. If the number of data files is below a threshold (e.g., N_{min}), it triggers the data mining procedure.

Topic-Oriented Data Mining For each topic m , an LLM agent generates a diverse set of Q search queries $\{q_1, \dots, q_Q\}$ strongly related to m and fetches up to R_{max} results per query using the web search tool initiated in the sandbox. The search results are organized and saved into several structured JSON files d_m with titles, main contents, section headings and summaries in a database D_m .

Stage2: Executable Tool Synthesis Given database D_m , we generate executable tools T_m through step-by-step synthesis enforcing strong constraints and validation to ensure quality, executability, and adherence to the agent interaction paradigm. For each data file $d \in D_m$ and topic m we construct the tool in python block with highly self-check constraints and validation mechanisms as below:

- **Standardized Interface.** Two top-level pre-defined functions including a list() and a get() function to query available items, categories with several topic-specific filtering parameters are expected. Each function must be decorated with `@mcp.tool(description="")` in a fixed format.

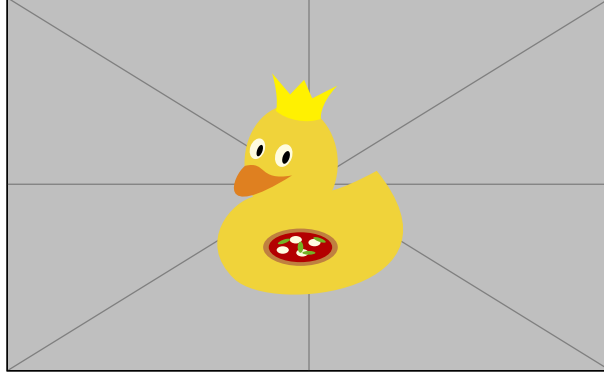


Figure 1: Overview.

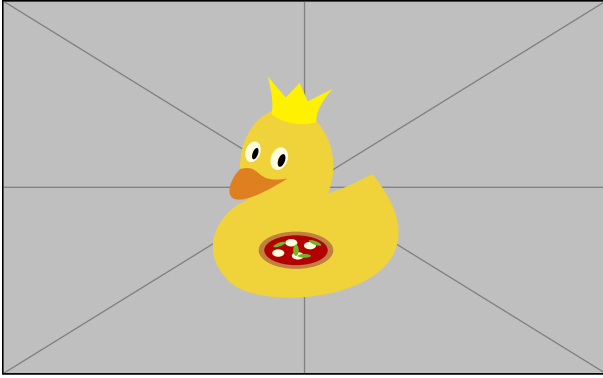


Figure 2: Taxonomy of environments of AgentGEM.

- **Data-Logic Decoupling.** Tools must read from the specified local file path instead of hard-code data within the function. A set of output keys are also inferred from the tool function as part of the metadata of tool specifications.
- **Layer Separation.** The tool used for final answer submission in tasks, is explicitly prohibited at this stage, maintaining a clear separation between the environment tool layer and the task layer.
- **Automated Validation.** The model is asked to verify that file paths in the generated code truly exist which can be imported. Besides, no top-level imports, helper functions, classes, or redefinitions of runtime objects are allowed in the current stage. After that, a lightweight smoke test is implemented to confirm that the return of two types of functions is non-empty and correctly structured results with sample parameters.

After the above-mentioned steps, the validated executable tool blocks are imported and registered

into the sandbox with tool specifications making it available for agent interactions.

Stage3: Verifiable Task Builder With environment established, the current module synthesizes challenging, multi-step tasks $X_m = x_1, x_2, \dots$. Each task x is a tuple containing a natural language query q , a submission schema f_{submit} , a ground-truth solution code sol , and a verification code ver .

Task Description Generation An LLM generates an initial task description q (default with difficulty level 1) given the topic m , D_m and T_m without mentioning tool names, file names, or specific data sources.

Submission Schema Generation In a separate LLM call, the system generates a JSON schema defining the exact structure of the final answer by decoupling the problem description generation with format structure to avoid mutual contamination conditioned on the task content.

Multi-Difficulty Refinement To iteratively refine the task with various difficulties, the LLM generate q_d with difficulty levels $d > 1$ with multi-condition limitations and joint reasoning and aggregation across data segments along with a corresponding f_{submit} .

Solution and Verification Code Generation This is the core step that ensures task verifiability.

3.1.2 Web Search Agent

3.1.3 CLI Agent

3.2 Cascaded Self-Evolving Agent Training

To mitigate this issue occurred in mixed agentic RL, we propose a two-stage self-evolving agent training method.

Table 1: Statistics of synthesized environments and tasks generated by AgentGEM.

Environment	# Tasks	Task Format	Tool Registry	Reasoning Depth	Verification
-------------	---------	-------------	---------------	-----------------	--------------

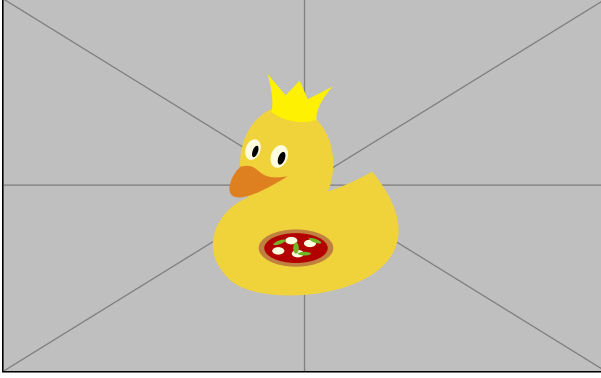


Figure 3: AgentGEM training recipe.

4 Experiments

4.1 Experiment Setup

Benchmarks. We assess the generalization of agents on unseen tasks using three types of benchmarks including (1) General Tool Use: VitaBench (He et al., 2025), τ^2 -Bench (Barres et al., 2025), and BFCL (v4) (Patil et al., 2025); (2) Web Search QA: MedQA (Jin et al., 2021), GPQA (Rein et al., 2024), and Bamboogle (Press et al., 2023); and (3) Software Engineering Problems: SWE-Bench (Jimenez et al., 2024) and Terminal-Bench 2.0 (Merrill et al., 2026). **Training Setup.** We use Qwen3-4B-Thinking-2507 (Yang et al., 2025a) as the base model and employ veRL (Sheng et al., 2024) and rLLM (Tan et al., 2025) for agentic RL post-training. We set train and mini batch sizes to 64 for strict on-policy updates, and a rollout size of 16 per training prompt. **Rollout & Inference Setup.** We employ random sampling (temperature $\tau = 0.6$ and $top-p = 0.95$ with a maximum output length of 2,048. We perform eight runs per test set and report the mean and standard deviation of the performance. **Evaluation Metrics.** All tasks in the work are evaluated using the algebraic mean

of hard exact match, which is also equivalent to pass@1 accuracy under strict extraction and comparison. **Compared Baselines.** We include five baselines including $\dots$ for comparison purposes.

4.2 AgentGEM Displays Consistent Effectiveness

We report the results of AgentGEM averaged over 8 runs, where we sampled trajectories independently for each benchmark in each run.

General Tool Use.

Web Search QA.

Software Engineering.

4.3 Scaling Effect of Synthetic Environments

5 Analyses

5.1 Impact of Environment Mixing Recipe

5.2 Plug-and-Play for General Agentic RL

5.3 Quality and Diversity of Environments

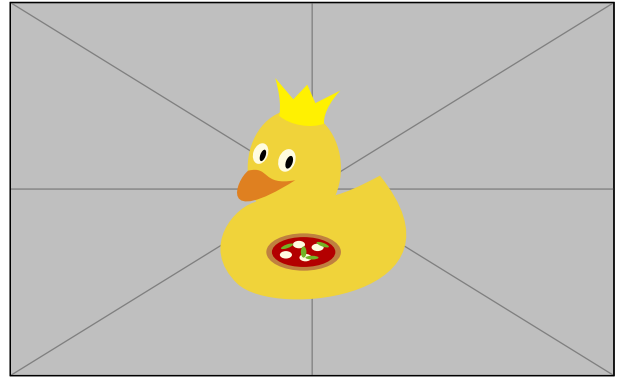


Figure 4: Training dynamics of AgentGEM-4B.

Table 2: Comparison with all baselines on eight benchmarks. AgentGEM outperforms most of other methods. Average is the macro mean across all samples of tasks.

Method	Tool Use		Conversation		Scientific QA		SWE / CLI		Avg. Results
	BFCL-v4	ACEBench	τ^2 -Bench	VitaBench	GPQA	MedQA	SWE-bench	TB-2.0	
ENVIRONMENT SCALING									
Toucan (Xu et al., 2025)									
Simia-Tau (Li et al., 2025)									
RLVE (Zeng et al., 2026)									
AWM (Wang et al., 2026)									
EnvScaler (Song et al., 2026)									
IMITATION LEARNING									
SFT w/o Trajectory (Wei et al., 2022)									
SFT w/ Trajectory (Yang et al., 2025b)									
ADVANCED RLVRs									
GRPO (Shao et al., 2024)									
StarPO (Wang et al., 2025)									
ARPO (Dong et al., 2026)									
STATIC CURRICULUM									
Data-balance RL (Parashar et al., 2026)									
Easy-to-hard RL (Parashar et al., 2026)									
ADARFT (Shi et al., 2025)									
AGENTGEM (Ours)									
AGENTGEM-4B									
AGENTGEM-8B									

Agent	Pass@1	Pass@32	Pass^32
AGENTGEM			
+ w/o CLI Env			
+ w/o MCP Env			
+ w/o WebQA Env			

Table 3: Ablation on environments of AgentGEM.

6 Related Work

Language agents often interleave internal deliberation with external actions. ReAct introduces a prompting-based reasoning-acting loop that improves interpretability and enables agents to incorporate tool observations into subsequent decisions (Yao et al., 2023b). Modular “LM + tools” architectures similarly combine language models with external modules (retrieval, symbolic solvers, calculators, or APIs), as exemplified by MRKL systems (Karpas et al., 2022) and program-aided methods such as PAL (Gao et al., 2023). Search-based deliberation over intermediate reasoning states has also been studied; Tree-of-Thoughts explores multiple candidate reasoning branches and selects among them using heuristic evaluation (Yao et al., 2023a). While these methods improve planning and reduce some failure modes, they typically rely on either hand-crafted prompting, fixed tool sets, or task-specific evalu-

ation heuristics, and they do not directly address scalable *environment* supply with reliable, automatic rewards.

Learning paradigms for tool use range from self-supervised trace construction to reinforcement learning. Toolformer proposes self-supervised generation of tool-call annotations so that models learn when and how to invoke tools (Schick et al., 2023). Gorilla targets accurate API selection and argument construction by training on API documentation and emphasizing generalization to unseen APIs (Patil et al., 2024). ToolLLM scales tool learning to large API collections and studies instruction and data construction for tool mastery (Qin et al., 2024). More recently, Tool Zero explores pure RL from scratch for tool-augmented LMs, highlighting the promise of RL but also the need for stable environments and dependable reward computation (Zeng et al., 2025). In parallel, improvements in open-source base model DeepSeek-V3.2 provides stronger agent backbones and make large-scale experimentation more accessible (DeepSeek-AI et al., 2025). However, stronger base models alone do not resolve the central bottleneck for agentic RL: the scarcity of interactive environments with grounded observations and non-trivial, execution-checked success criteria.

AgentGEM complements these lines by focusing on the *environment generation* layer. It operationalizes schema-described tool interfaces and couples generation with sandboxed execution, targeted repair, and explicit verifiers, enabling scalable production of interaction traces where rewards are computed from machine-checkable acceptance procedures rather than from brittle string matching or underspecified heuristics.

7 Conclusion

We introduced an effective and scalable method called AgentGEM, and observed that it significantly improves success rate in a range of agentic tasks, across four kinds of benchmarks with varying task desiderata. Beyond accuracy gains, AgentGEM also demonstrates that . . .

Limitations

Ethical Considerations

References

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. *tau²-bench: Evaluating conversational agents in a dual-control environment*. *arXiv preprint arXiv:2506.07982*.

DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, and 245 others. 2025. *Deepseek-v3.2: Pushing the frontier of open large language models*. *Preprint*, arXiv:2512.02556.

Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2026. *Agentic reinforced policy optimization*. In *The Fourteenth International Conference on Learning Representations*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. *PAL: Program-aided language models*. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Wei He, Yueqing Sun, Hongyan Hao, Xueyuan Hao, Zhikang Xia, Qi Gu, Chengcheng Han, Dengchang Zhao, Hui Su, Kefeng Zhang, and 1 others. 2025. *Vitabench: Benchmarking llm agents with versatile interactive tasks in real-world applications*. *arXiv preprint arXiv:2509.26490*.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. *SWE-bench: Can language models resolve real-world github issues?* In *The Twelfth International Conference on Learning Representations*.

Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. *What disease does this patient have? a large-scale open domain question answering dataset from medical exams*. *Applied Sciences*, 11(14):6421.

Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, Yoav Shoham, Hofit Bata, Yoav Levine, Kevin Leyton-Brown, and 1 others. 2022. *Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning*. *arXiv preprint arXiv:2205.00445*.

Yuetai Li, Huseyin A Inan, Xiang Yue, Wei-Ning Chen, Lukas Wutschitz, Janardhan Kulkarni, Radha Poovendran, Robert Sim, and Saravan Rajmohan. 2025. *Simulating environments with reasoning models for agent training*. *arXiv preprint arXiv:2511.01824*.

Mike A Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, and 65 others. 2026. *Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces*. In *The Fourteenth International Conference on Learning Representations*.

Shubham Parashar, Shurui Gui, Xiner Li, Hongyi Ling, Sushil Vemuri, Blake Olson, Eric Li, Yu Zhang, James Caverlee, Dileep Kalathil, and Shuiwang Ji. 2026. *Curriculum reinforcement learning from easy to hard tasks improves LLM reasoning*. In *The Fourteenth International Conference on Learning Representations*.

Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. *The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models*. In *Forty-second International Conference on Machine Learning*.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. *Gorilla: Large language model connected with massive apis*. In *Advances in Neural Information Processing Systems*, volume 37, pages 126544–126565. Curran Associates, Inc.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2023. *Measuring and narrowing the compositionality gap in language models*. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711.

437	Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan	agents via multi-turn reinforcement learning. <i>arXiv</i>	494
438	Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang,	<i>preprint arXiv:2504.20073</i> .	495
439	Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian,		
440	Ruobing Xie, Jie Zhou, Mark Gerstein, dahai li,	Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu,	496
441	Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM:	Adams Wei Yu, Brian Lester, Nan Du, Andrew M.	497
442	Facilitating large language models to master 16000+	Dai, and Quoc V Le. 2022. Finetuned language	498
443	real-world APIs . In <i>The Twelfth International Con-</i>	models are zero-shot learners . In <i>International Con-</i>	499
444	<i>ference on Learning Representations</i> .	<i>ference on Learning Representations</i> .	500
445	David Rein, Betty Li Hou, Asa Cooper Stickland,	Zhangchen Xu, Adriana Meza Soria, Shawn Tan,	501
446	Jackson Petty, Richard Yuanzhe Pang, Julien Di-	Anurag Roy, Ashish Sunil Agrawal, Radha Pooven-	502
447	rani, Julian Michael, and Samuel R. Bowman. 2024.	dran, and Rameswar Panda. 2025. Toucan: Synthe-	503
448	GPQA: A graduate-level google-proof q&a bench-	sizing 1.5 m tool-agentic data from real-world mcp	504
449	mark . In <i>First Conference on Language Modeling</i> .	environments. <i>arXiv preprint arXiv:2510.01179</i> .	505
450	Timo Schick, Jane Dwivedi-Yu, Roberto Dessi,	An Yang, Anfeng Li, Baosong Yang, Beichen Zhang,	506
451	Roberta Raileanu, Maria Lomeli, Eric Hambro,	Binyuan Hui, Bo Zheng, Bowen Yu, Chang	507
452	Luke Zettlemoyer, Nicola Cancedda, and Thomas	Gao, Chengen Huang, Chenxu Lv, and 1 others.	508
453	Scialom. 2023. Toolformer: Language models can	2025a. Qwen3 technical report. <i>arXiv preprint</i>	509
454	teach themselves to use tools . In <i>Advances in Neural</i>	<i>arXiv:2505.09388</i> .	510
455	<i>Information Processing Systems</i> , volume 36, pages		
456	68539–68551. Curran Associates, Inc.	Shiming Yang, Yuxuan Tong, Xinyao Niu, Graham	511
457	Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu,	Neubig, and Xiang Yue. 2025b. Demystifying long	512
458	Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan	chain-of-thought reasoning . In <i>Forty-second Inter-</i>	513
459	Zhang, YK Li, Yang Wu, and 1 others. 2024.	<i>national Conference on Machine Learning</i> .	514
460	Deepseekmath: Pushing the limits of mathematical		
461	reasoning in open language models. <i>arXiv preprint</i>	Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran,	515
462	<i>arXiv:2402.03300</i> .	Tom Griffiths, Yuan Cao, and Karthik Narasimhan.	516
463	Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin	2023a. Tree of thoughts: Deliberate problem solv-	517
464	Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin	ing with large language models . In <i>Advances in</i>	518
465	Lin, and Chuan Wu. 2024. Hybridflow: A flexible	<i>Neural Information Processing Systems</i> , volume 36,	519
466	and efficient rlhf framework. <i>arXiv preprint arXiv:</i>	pages 11809–11822. Curran Associates, Inc.	520
467	<i>2409.19256</i> .		
468	Taiwei Shi, Yiyang Wu, Linxin Song, Tianyi Zhou, and	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak	521
469	Jieyu Zhao. 2025. Efficient reinforcement finetun-	Shafran, Karthik R Narasimhan, and Yuan Cao.	522
470	ing via adaptive curriculum learning. <i>arXiv preprint</i>	2023b. React: Synergizing reasoning and acting	523
471	<i>arXiv:2504.05520</i> .	in language models . In <i>The Eleventh International</i>	524
472	Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao	<i>Conference on Learning Representations</i> .	525
473	Zhu, Ji-Rong Wen, and Zhicheng Dou. 2026. Envs-		
474	caler: Scaling tool-interactive environments for llm	Yirong Zeng, Xiao Ding, Yutai Hou, Yuxian Wang,	526
475	agent via programmatic synthesis. <i>arXiv preprint</i>	Li Du, Juyi Dai, Qiuyang Ding, Duyu Tang, Dan-	527
476	<i>arXiv:2601.05808</i> .	dan Tu, Weiwen Liu, Bing Qin, and Ting Liu. 2025.	528
477	Sijun Tan, Michael Luo, Colin Cai, Tarun Venkat,	Tool zero: Training tool-augmented LLMs via pure	529
478	Kyle Montgomery, Aaron Hao, Tianhao Wu, Arnab	RL from scratch . In <i>Findings of the Association</i>	530
479	Balyan, Manan Roongta, Chenguang Wang, Li Er-	<i>for Computational Linguistics: EMNLP 2025</i> , pages	531
480	ran Li, Raluca Ada Popa, and Ion Stoica. 2025. rllm:	9135–9147, Suzhou, China. Association for Compu-	532
481	A framework for post-training language agents.	tational Linguistics.	533
482	https://pretty-radio-b75.notion.site/	Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Li-	534
483	rLLM-A-Framework-for-Post-Training-Language-Agents	fanyuan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting	535
484	Notion Blog.	Li, Jacqueline He, Runlong Zhou, Tong Chen,	536
485	Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Si-	Chenyang Zhao, Yulia Tsvetkov, Simon Shaolei Du,	537
486	wei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He.	Natasha Jaques, Hao Peng, Pang Wei Koh, and Han-	538
487	2026. Agent world model: Infinity synthetic envi-	nam. 2025. R1: Reinforcing reasoning for	539
488	ronments for agentic reinforcement learning. <i>arXiv</i>	reinforcement learning for language models with adaptive	540
489	<i>preprint arXiv:2602.10090</i> .	verifiable environments . In <i>International Confer-</i>	541
490	Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue	<i>ence on Machine Learning (ICML)</i> .	542
491	Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan		
492	Yu, Minh Nhat Nguyen, Licheng Liu, and 1 others.	A Appendix	543
493	2025. Ragen: Understanding self-evolution in llm	This is an appendix.	544